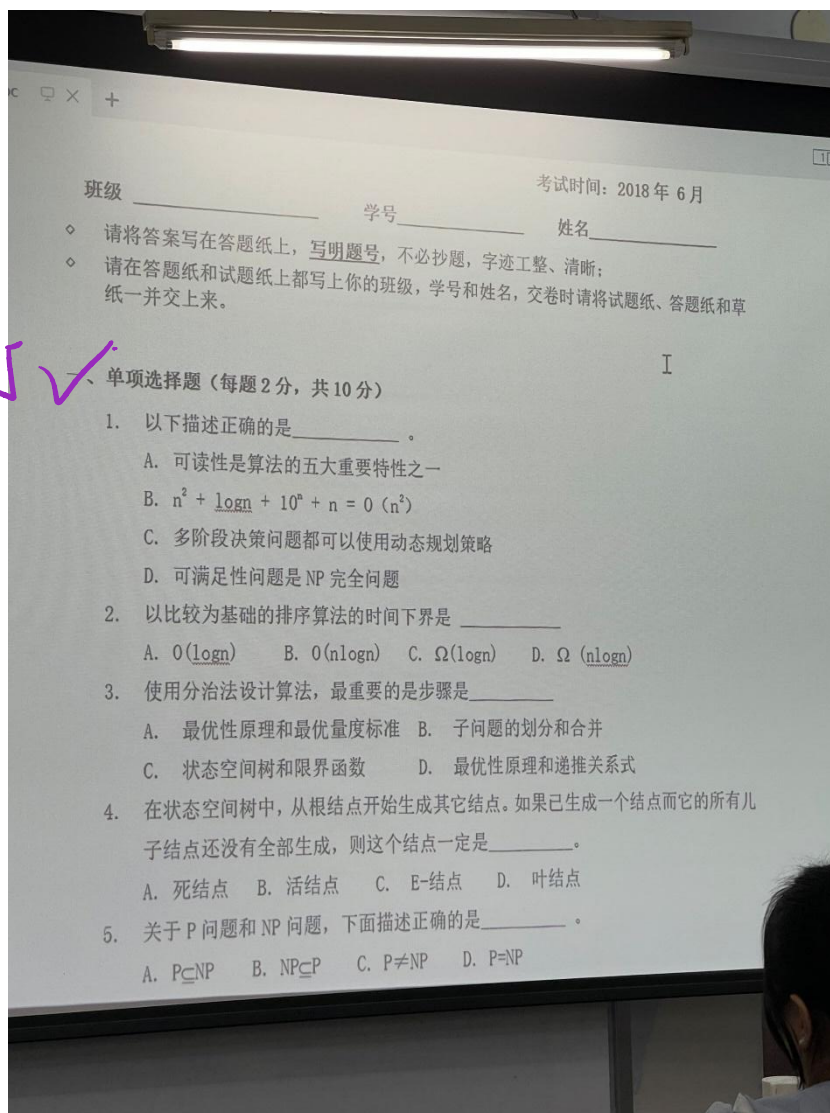




考试时间: 2018 年 6 月

班级 _____

学号 _____

姓名 _____

- ◇ 请将答案写在答题纸上, 写明题号, 不必抄题, 字迹工整、清晰;
- ◇ 请在答题纸和试题纸上都写上你的班级, 学号和姓名, 交卷时请将试题纸、答题纸和草稿纸一并交上来。

一、单项选择题 (每题 2 分, 共 10 分)

I

1. 以下描述正确的是 _____。
A. 可读性是算法的五大重要特性之一
B. $n^2 + \log n + 10^6 + n = O(n^2)$
C. 多阶段决策问题都可以使用动态规划策略
D. 可满足性问题是 NP 完全问题
2. 以比较为基础的排序算法的时间下界是 _____
A. $O(\log n)$ B. $O(n \log n)$ C. $\Omega(\log n)$ D. $\Omega(n \log n)$
3. 使用分治法设计算法, 最重要的是步骤是 _____
A. 最优性原理和最优量度标准 B. 子问题的划分和合并
C. 状态空间树和限界函数 D. 最优性原理和递推关系式
4. 在状态空间树中, 从根结点开始生成其它结点。如果已生成一个结点而它的所有儿子结点还没有全部生成, 则这个结点一定是 _____。
A. 死结点 B. 活结点 C. E-结点 D. 叶结点
5. 关于 P 问题和 NP 问题, 下面描述正确的是 _____。
A. $P \subseteq NP$ B. $NP \subseteq P$ C. $P \neq NP$ D. $P = NP$

5. 关于 P 问题和 NP 问题, 下面描述正确的是_____。

- A. $P \subseteq NP$ B. $NP \subseteq P$ C. $P \neq NP$ D. $P = NP$

4 ✓ 简答题 (每题 5 分, 共 20 分)

1. 求解以下递推关系式 (a 为常数项)

$$T(n) = \begin{cases} a & n \leq 2 \\ 2T(n-1) + 1 & n > 2 \end{cases}$$

2. 什么是最优性原理? 举出 1 个最优性原理不成立的例子。

3. 已知 4 个正数: $(w_1, w_2, w_3, w_4) = (5, 6, 11, 17)$ 和 $M=28$, 画出该子集和数问题不定长元组的状态树, 并且按宽度优先检索顺序对其节点编号, 给出答案状态的编号。

4. 简述回溯法和分支-限界法的主要区别。

2 ✓ 证明题 (15 分)

1. (7 分) 设 $f(n) \geq 0$, $g(n) \geq 0$, $g(n) = \Omega(f(n))$, 证明 $\Omega(f(n)) + \Omega(g(n)) = \Omega(f(n))$

2. (8 分) 已知 $p_1/w_1 \geq p_2/w_2 \geq \dots \geq p_n/w_n$, 证明贪心算法 GREEDY-KNAP

2017-2018 学年 第 2 学期

2018 级《算法设计与分析》试题 A 卷答案

考试时间: 2018 年 6 月

一、单项选择题 (每题 2 分, 共 10 分)

DDBBA | I

二、简答题 (每题 5 分, 共 20 分)

1. 求解以下递推关系式 (a 为常数项)

$$T(n) = \begin{cases} a & n \leq 2 \\ 2T(n-1) + 1 & n > 2 \end{cases}$$

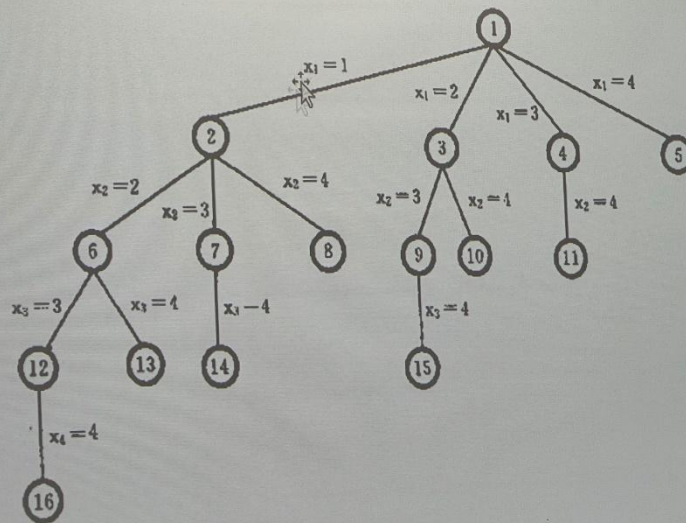
解: $T(n) = 2T(n-1) + 1 = 2(2T(n-2) + 1) + 1 = \dots = 2^n T(1) + 2n - 1 = 0(2^n)$

2. 什么是最优性原理? 举出 1 个最优性原理不成立的例子。

答: 无论过程的初始状态和初始决策是什么, 其余的决策必须相对于初始决策所产生的状态构成一个最优决策序列。比如: 多段图问题: 路径和改为路径乘积并允许存在权值为负数的边

(5, 6, 11, 17) 和 $M=28$, 画出该子集和数问题不定长

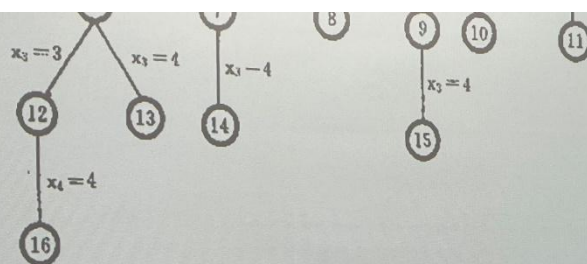
元组的状态树，并且按宽度优先检索顺序对其节点编号，给出答案状态的编号。
答：



答案状态为 11 和 12

4. 简述回溯法和分支-限界法的主要区别。

答：回溯法是指在状态空间树上使用限界函数进行深度优先节点生成方法，在生成当前 E-节点的全部儿子之后再生成其它活结点的儿子。



答案状态为 11 和 12

4. 简述回溯法和分支-限界法的主要区别。

答：回溯法是指在状态空间树上使用限界函数进行深度优先节点生成的方法。

分枝-限界法是在生成当前 E-节点的全部儿子之后再生成其它活结点的儿子且用限界函数帮助避免生成不包含答案节点子树的状态空间检索方法。

两者区别在于分支限界法的某个活结点一旦成为 E-节点，则其一直保持到变成死结点为止，而回溯法中某个活结点成为 E-节点后，仍可能还原成为活结点而使其它节点成为 E-节点。

三、证明题 (15 分)

1. (5 分) 设 $f(n) \geq 0$, $g(n) \geq 0$, $g(n) = \Omega(f(n))$, 证明 $\Omega(f(n)) + \Omega(g(n)) = \Omega(f(n))$

2. 什么是最优性原理？举出 1 个最优性原理不成立的例子。
3. 已知 4 个正数：(w_1, w_2, w_3, w_4) = (5, 6, 11, 17) 和 $M=28$ ，画出该子集和数问题不定长元组的状态树，并且按宽度优先检索顺序对其节点编号，给出答案状态的编号。
4. 简述回溯法和分支-限界法的主要区别。

三、证明题 (15 分)

1. (7 分) 设 $f(n) \geq 0, g(n) \geq 0, g(n) = \Omega(f(n))$ ，证明 $\Omega(f(n)) + \Omega(g(n)) = \Omega(f(n))$ 。
2. (8 分) 已知 $p_1/w_1 \geq p_2/w_2 \geq \dots \geq p_n/w_n$ ，证明贪心算法 GREEDY-KNAPSACK 对于任意给定的背包问题实例生成一个最优解。

四、计算题 (每题 10 分，共 40 分)

1. 求解带有限期的作业排序问题：作业数 $n=7$
 $(p_1, p_2, \dots, p_7) = (8, 12, 7, 5, 11, 6, 9)$
 $(d_1, d_2, \dots, d_7) = (1, 3, 2, 5, 5, 3, 2)$

分支-限界法是在生成当前 E-节点的全部儿子之后再生成其它活结点的儿子且用限界函数帮助避免生成不包含答案节点子树的状态空间检索方法。
 两者区别在于分支限界法的某个活结点一旦成为 E-节点，则其一直保持到变成死结点为止，而回溯法中某个活结点成为 E-节点后，仍可能还原成为活结点而使其它节点成为 E-节点。

三、证明题 (15 分)

1. (5 分) 设 $f(n) \geq 0, g(n) \geq 0, g(n) = \Omega(f(n))$ ，证明 $\Omega(f(n)) + \Omega(g(n)) = \Omega(f(n))$ 。
 证明：
 设 $f_1(n) = \Omega(f(n))$ ，则存在常数 n_1, c_1 ，使得 $n \geq n_1$ 时，有 $f_1(n) \geq c_1 f(n)$ 。
 设 $f_2(n) = \Omega(g(n))$ ，则存在常数 n_2, c_2 ，使得 $n \geq n_2$ 时，有 $f_2(n) \geq c_2 g(n)$ ；
 由 $g(n) = \Omega(f(n))$ ，可知存在正常数 n_3, c_3 ；使得 $n \geq n_3$ 时，有 $g(n) \geq c_3 f(n)$ ；
 当 $n \geq \max\{n_1, n_2, n_3\}$ 时，

$$\Omega(f(n)) + \Omega(g(n)) = f_1(n) + f_2(n) \geq c_1 f(n) + c_2 g(n) \geq c_1 f(n) + c_2 c_3 f(n) = (c_1 + c_2 c_3) f(n),$$

 令 $c_0 = (c_1 + c_2 c_3)$ ， $n_0 = \max\{n_1, n_2, n_3\}$ ，命题得证。

2. (10 分) 如果 $p_1/w_1 \geq p_2/w_2 \geq \dots \geq p_n/w_n$ ，则贪心算法 GREEDY-KNAPSACK 对于任意给定的背包问题实例生成一个最优解。

四、计算题（每题 10 分，共 40 分）

1. 求解带有限期的作业排序问题：作业数 $n=7$

$$(p_1, p_2, \dots, p_7) = (8, 12, 7, 5, 11, 6, 9)$$

$$(d_1, d_2, \dots, d_7) = (1, 3, 2, 5, 5, 3, 2)$$

采用算法 5.5 作业排序的更快算法，求解上述作业排序问题的最优解，要求按步骤运行并给出

- (1) 变化中的 $F(i)$ 值 (2 分)
- (2) 集合树的变化情况 (5 分)
- (3) 最优解 (2 分)
- (4) 效益值 (1 分)

2. 对下述 0/1 背包问题， $n=5, M=15$ ，物品对应的效益值和重量如下：

物品 i	1	2	3	4	5
效益 $P(i)$	1	9	7	4	10
重量 $w(i)$	8	2	2	6	6

(4) 效益值 (1 分)

2. 对下述 0/1 背包问题， $n=5, M=15$ ，物品对应的效益值和重量如下：

物品 i	1	2	3	4	5
效益 $P(i)$	1	9	7	4	10
重量 $w(i)$	8	2	2	6	6

请用序偶对方法求解该问题。

- (1) 依次按步骤求 S^i ($i=0,1,2,3,4,5$) (6 分)
- (2) 给出最大效益值 (2 分)
- (3) 回溯求最优解 X (2 分)

3. 有 a, b 两台设备的流水线调度问题，设数组 $A(n), B(n)$ 分别存储 a, b 的时间， $n=5$,

- (2) 给出最大效益值 (2分)
- (3) 回溯求最优解 X (2分)

3. 有 a, b 两台设备的流水线调度问题, 设数组 $A(n), B(n)$ 分别存储 a, b 的任务完成时间, $n=5$,

作业 i	1	2	3	4	5
$A(i)$	30	60	90	80	40
$B(i)$	100	20	70	10	50

- (1) 给出这五个作业的非抢先最优调度序列 (5分)
- (2) 画出调度图。(3分)
- (3) 给出此调度的完成时间 $F(S)$ 。(2分)

4. 考虑 9-谜问题, 给定初始状态和目标状态描述如下:

5	6	1
3	8	
4	2	7

初始状态

3	5	1
6		8
4	2	7

目标状态

要求通过空格的移动, 将初始状态逐步变为目标状态。假设空格按照“上右下左”的次序移动。

请用 LC 检索法解决该问题, 采用函数 $\hat{c}(x) = f(x) + g(x)$ 作为成本估计值的函数, 其中 $f(x)$ 是根到结点 x 路径的长度, $g(x)$ 是节点 x 中不在其目标位的非空白牌数目。要求给出求解过程中生成的状态空间树, 并标出树中每个状态的 $\hat{c}$ 值 (10分)。

五、算法题 (共15分)

所谓单峰数组 $A[n]$, 是指数组元素值互不相同, 数组元素先递增再递减, 即 $A[1] < A[2] < \dots < A[p-1] < A[p]$ 且 $A[p] > A[p+1] > \dots > A[n]$ 。下标 p ($1 < p < n$) 称为单峰位置。

例如, 数组 $A[5] = (1, 2, 5, 4, 3)$, 因 $A[1] < A[2] < A[3]$ 且 $A[3] > A[4] > A[5]$, 故 $p=3$ 是一个单峰位置。

(2) 画出调度图。(3分)

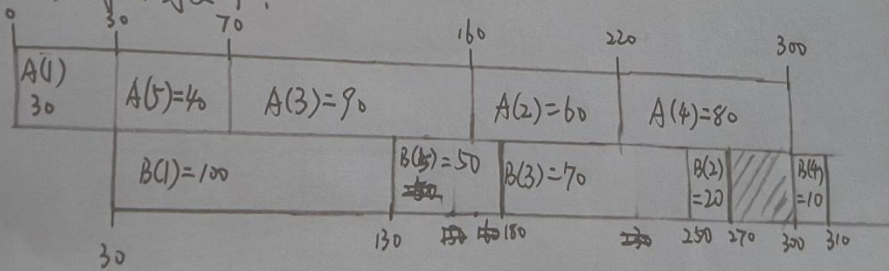
(3) 给出此调度的完成时间 $F(S)$ 。(2分)

3. 解：将 $A(i), B(i)$ 从小到大排序：

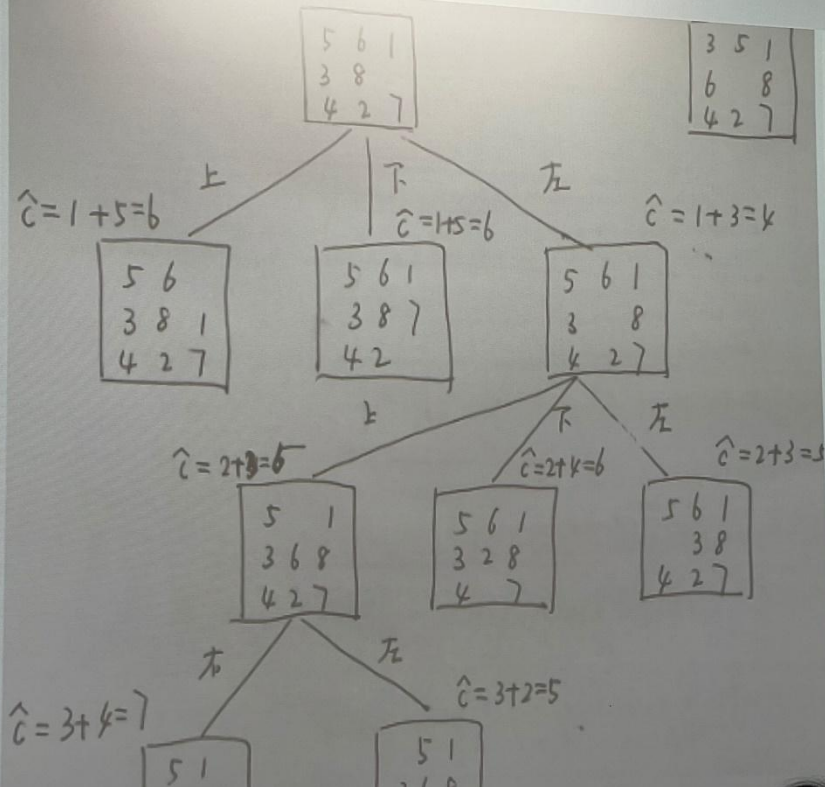
$B(4), B(2), A(1), A(5), B(5), A(2), B(3), A(4), A(3), B(1)$

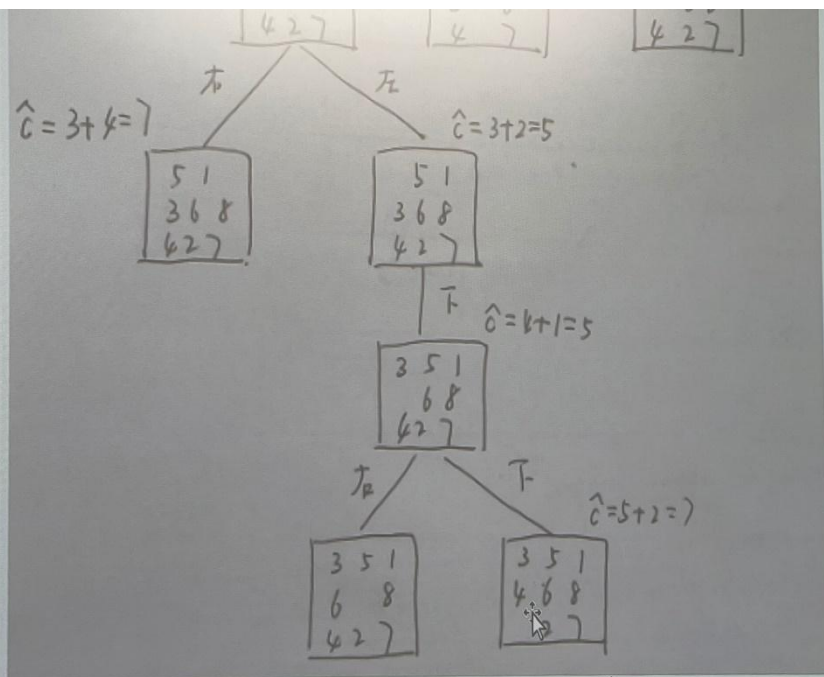
(一) 故调度序列为 1, 5, 3, 2, 4

(二) 调度图如下：



(三) 调度完成时间为 310。





五、算法题（共15分）

所谓单峰数组 $A[n]$ ，是指数组元素值互不相同，数组元素先递增再递减，即 $A[1] < A[2] \dots < A[p-1] < A[p]$ 且 $A[p] > A[p+1] > \dots > A[n]$ 。下标 p ($1 < p < n$) 被称为单峰位置。例如：数组 $A[5] = (1, 2, 5, 4, 3)$ ，因 $A[1] < A[2] < A[3]$ 且 $A[3] > A[4] > A[5]$ ，故数组 $A[5]$ 就是一个单峰数组，且单峰位置为 3。

已知数组 $A[n]$ 为单峰数组，但不知具体的单峰位置，现要求设计算法找到其单峰位置 p 。

请回答如下问题：

- (1) 若通过对数组从前往后进行顺序扫描的方案来确定单峰位置 p ，请问最好情况下的时间复杂度是多少？（1分）最坏情况下的时间复杂度是多少？（1分）
 $O(n)$
- (2) 请给出用分治法解决该问题的算法描述（8分），所设计算法在最好情况下的时间复杂度是多少？（1分）最坏情况下的时间复杂度是多少？为什么？（4分）

(2) 请给出用分治法解决该问题的算法描述 (8 分), 所设计算法在最好情况下的时间复杂度是多少? (1 分) 最坏情况下的时间复杂度是多少? 为什么? (4 分) |

解:

(1) 最好情况下时间复杂度为 $O(1)$, 最坏情况下时间复杂度为 $O(n)$.

(2) 用分治法解决, 根据算法描述, 已知为单峰数组, 故可知 n 大于等于 3. $n \geq 3$

算法 `int findpeak(A[n])`

`Low=1, high=n, mid=[(low+high)/2]取地板`

`While low<high do`

`If A[mid-1]<A[mid] then`

`If A[mid]>A[mid+1] then return mid`

`Else low=mid`

`Endif`

`If A[mid-1]<A[mid] then`

`If A[mid]>A[mid+1] then return mid`

`Else low=mid`

`Endif`

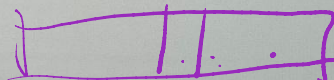
`Else high=mid`

`Endif`

`Repeat`

`End`

$\left\{ \begin{array}{l} \text{左} < \text{mid} > \text{右} \quad P = \text{mid} \\ \text{左} < \text{mid} < \text{右} \quad \text{low} = \text{mid} + 1, P \text{ 在右} \\ \text{左} > \text{mid} \quad P \text{ 在左, high} = \text{mid} - 1 \end{array} \right.$



算法最好情况下, 是第一个 `mid` 就是单峰位置, 所以时间复杂度为 $O(1)$.

最坏情况下, 需要对数组进行不断二分, 时间复杂度为对应二叉查找树的高度, 因此为

$O(\log n)$.